Training the continuous-saturation emulator in nGibbs

Ben Thyer

October 2, 2026

Abstract

This note describes how the continuous-saturation emulator (`ContinuousModel`) is trained, with an emphasis on how each term of the loss is evaluated on a mini-batch. The model is defined in `NN_continuous.py` and trained by `train_Upper_MELTS` in `trainer.py`. Table 1 is the key to all symbols.

1 Variables

Dimensions use the letter codes of the ML indexer (`src/ngibbs/config/README_MLIndexer.md`): P phases, VP compositionally variable phases, C components (endmembers of all phases), VC components of the variable phases, E elements and O oxides. Two further sizes are used here: F, the number of physical input features (e.g. pressure and temperature), and B, the number of rows in a mini-batch. Every quantity in Table 1 is per row; a batch stacks B rows. For emulator outputs, the hatted symbol is the prediction and the plain symbol is the ground truth from the thermodynamic model.

2 The forward model

The network sees $[u, b]$. An encoder produces a feature vector h, which is concatenated with the raw input. From it:

1. one softmax head per variable phase gives that phase's endmember fractions $\hat{x}$. Endmembers containing an element that is absent from b are given zero probability;
2. one small network per phase, which also sees $\hat{x}$, gives the affinity g;
3. the abundance is the rectified affinity,

$$\hat{n} = \max(g, 0). \tag{1}$$

Because $\max(g, 0)$ is continuous at $g = 0$, a phase leaving the assemblage is a kink in $\hat{n}$, not a jump. (In the code g first passes through a leaky ReLU and is then clamped at zero; the result is identical.)

Component moles and the reconstructed bulk follow from stoichiometry:

$$\hat{m} = (\hat{x}V + b) \circ (\hat{n}M), \tag{2}$$

$$\hat{b} = \hat{m}A. \tag{3}$$

Here $\circ$ is the element-wise product, so $\hat{m}$ is each component's phase abundance times its fraction in that phase. Mass balance (adjusting $\hat{m}$ until $\hat{b} = b$ exactly) is applied only at inference, by `NN_MELTS.apply_mass_balance`, and is not part of the trained network.

Prediction	Truth	Dim.	Meaning
Inputs			
	u	F	Physical state (pressure, temperature, ...), min–max normalised to $[0, 1]$
$\hat{b}$	b	E	Bulk composition in element moles, normalised to sum to 1. $\hat{b}$ is the bulk reconstruction $\hat{b} = \hat{m}A$
Emulator outputs			
$\hat{n}$	n	P	Phase abundance (moles); 0 for an absent phase
$\hat{x}$	x	VC	Endmember mole fractions of each variable phase (sum to 1 within a phase: $\sum x_i = s_i \lvert 1 - b \rvert$)
$\hat{m}$	m	C	Extensive endmember components/species in moles; $\hat{m} = (\hat{x}V + b) \circ (\hat{n}M)$
$\hat{s}$	s	P	Phase presence, 1 if present and 0 if absent; $s = [n > 0]$
Quantities used only in training			
g	–	P	Phase affinity/abundance: the raw output of each phase's mole head; $\hat{n} = \max(g, 0)$
$\tilde{n}$	–	P	Smoothed, positive, non-zero abundance used to evaluate mole loss; $\tilde{n}(g, T)$: Eq. (9)
–	$\bar{s}$	VC	Phase Presence s projected to phases' endmembers (chemistry mask) $\bar{s} = s\,M\,V^\top \in \{0, 1\}$
Training constants			
T_0		P	Reference abundance scale per phase (5th percentile of present abundances)
T		P	Annealed smoothing temperature; $T = a\,T_0$
a		1	Annealing factor, decreased linearly over an episode
w		P	Phase weights
ω		VC	Endmember weights; $\omega = w\,M\,V^\top$
$\lambda_s, \lambda_x, \lambda_n, \lambda_b$		1	Weights for the binary phase presence, chemistry, molar, and bulk loss terms
Fixed stoichiometric/projection matrices			
A		C×E	Element moles per component mole (`compToEl`)
M		P×C	Which components belong to which phase; $M \in \{0, 1\}$ (`phaseToCompMap`)
V		VC×C	Indexing matrix that places the VC components into the full component list; $M \in \{0, 1\}$ (`variedToAllComp`)
b		C	which components belong to fixed-composition (pure) phases; $b \in \{0, 1\}$ (`fixed_phaseToCompMap`)

Table 1: Key to the symbols. "–" marks a quantity with no prediction or no ground-truth counterpart.

3 The training objective

For each mini-batch the trainer minimises

$$\boxed{L = 10^4\big(\lambda_s L_s + \lambda_x L_x + \lambda_n L_n + \lambda_b L_b\big)} \tag{4}$$

covering phase presence (L_s), chemistry (L_x), phase abundance (L_n) and bulk (L_b). The factor 10^4 keeps the loss large compared with the optimiser's ϵ. The bulk term is added only when $\lambda_b \neq 0$. Typical recipe values are $\lambda_x = 500$, $\lambda_n = 50$, λ_s between 0.002 and 0.02, and $\lambda_b = 0$ while tuning, rising to 50 for the final joint training. The same function (`_upper_loss`) computes the training loss and the validation loss, so a model is always scored on the objective it was trained on.

In every term below, sums ($\sum$) run over all B rows of the batch as well as over the index shown, and each term is a weighted mean: the weighted sum of errors divided by the sum of the weights.

3.1 The error function ρ

Chemistry, abundance and bulk use the symmetric relative squared error (`symmetric_rel_l2`),

$$\rho(\hat{y}, y) = \frac{(\hat{y} - y)^2}{|\hat{y}| + |y|}, \tag{5}$$

with the denominator floored at 10^{-6}. The type is chosen separately for each term by `loss_config.<term>.type` (`symmetric_rel_l1` and `huber` are also available). Two limits explain its behaviour:

$$y = 0: \qquad \rho(\hat{y}, 0) = |\hat{y}|, \tag{6}$$

$$\hat{y} \approx y: \qquad \rho(\hat{y}, y) \approx \frac{(\hat{y} - y)^2}{2y}. \tag{7}$$

For a target of zero it acts like an absolute (L_1) error, a constant pull toward zero. Near a non-zero target it is a squared error divided by the target, so a small abundance counts as much as a large one in relative terms.

3.2 Chemistry loss L_x

Only phases that are present in the ground truth have a composition, so the predicted fractions are masked by $\bar{s}$ before comparison and the mean runs over present endmembers only:

$$L_x = \frac{\sum_k \bar{s}_k\, \omega_k\, \rho(\bar{s}_k \hat{x}_k,\ x_k)}{\sum_k \bar{s}_k\, \omega_k}, \qquad k = 1, \ldots, \mathrm{VC}. \tag{8}$$

3.3 Abundance loss L_n

Every phase is supervised, including absent ones (target $n = 0$): learning where an abundance reaches zero is the point of the continuous model. The loss is not computed on $\hat{n}$ itself but on a smoothed version of it controlled by the boundary temperature T (Section 4):

$$\tilde{n} = T\,\mathrm{softplus}(g/T) = T \ln\big(1 + e^{g/T}\big), \tag{9}$$

and

$$L_n = \frac{\sum_j w_j\, \rho(\tilde{n}_j, n_j)}{\sum_j w_j}, \qquad j = 1, \ldots, \mathrm{P}. \tag{10}$$

$\tilde{n}$ is a smooth stand-in for $\max(g, 0)$ with two useful properties:

$$\frac{d\tilde{n}}{dg} = \text{sigmoid}(g/T), \qquad 0 < \tilde{n} - \max(g, 0) \le T \ln 2. \tag{11}$$

The error relative to $\hat{n}$ is largest ($T \ln 2$) at $g = 0$, negligible once $|g| > 3T$, and shrinks as T is annealed. For an absent phase, Eq. (6) gives $\rho(\tilde{n}, 0) = \tilde{n}$, so the gradient on its affinity is sigmoid(g/T). This pushes g negative and fades to zero once the phase is confidently absent ($g \ll -T$), instead of holding it at the boundary $g = 0$.

3.4 Presence loss L_s

There is no separate presence head. Instead the affinity is also used as the logit of a binary cross-entropy against s:

$$L_s = \frac{\sum_j c_j \ \text{BCE}\big(5g_j/T_{0,j},\ s_j\big)}{\sum_j c_j}, \qquad c = \big(s\,w + (1 - s)\big)\,\big|\tanh(g/T)\big|, \tag{12}$$

with $\text{BCE}(z, s) = -s \ln \text{sigmoid}(z) - (1-s) \ln\big(1 - \text{sigmoid}(z)\big)$. Three choices are built into Eq. (12):

1. Phase weights apply only to the present examples ($s = 1$). Those are the examples a rare phase is short of; absent examples keep weight 1.
2. The logit is scaled by the fixed T_0, not the annealed T, so the classifier's confidence means the same thing throughout training: it reaches sigmoid$(5) \approx 0.99$ at $g = T_0$.
3. The weight $|\tanh(g/T)|$ uses the annealed T. It is zero at the boundary and close to 1 a few T away. It hands off with the abundance loss, whose gradient sigmoid(g/T) is largest near the boundary and fades far from it: L_n handles calibration close to $g = 0$, and L_s corrects samples that are confidently wrong far from it.

The weight c depends on g and is not detached, so it contributes to the gradient.

3.5 Bulk loss L_b

The reconstructed bulk (Eq. 3) is compared with the input bulk over the elements present in it:

$$L_b = \frac{\sum_{e:\, b_e > 0} \rho(\hat{b}_e, b_e)}{\sum_{e:\, b_e > 0} 1}, \qquad e = 1, \ldots, \text{E}. \tag{13}$$

This measures how far the raw predictions are from satisfying mass balance, which reduces the correction that has to be applied at inference. It is meaningless until the mole heads have been trained, so it is switched on only for the final joint training. $\hat{b}$ is built from the unsmoothed $\hat{n}$.

4 Boundary temperature

The reference scale T_0. T_0 sets, for each phase, how small an abundance is "small". It is the 5th percentile of that phase's abundance over the rows in which the phase is present:

$$T_{0,j} = \text{5th percentile of } \{\, n_j : n_j > 0 \,\}. \tag{14}$$

An abundance below $T_{0,j}$ is therefore smaller than in 95% of the rows where phase j is present. Because the training and validation sets are generated the same way, their T_0 values are essentially identical, so T_0 is computed on the smaller validation set, where it is cheaper. Across a typical

dataset T_0 spans about three orders of magnitude (roughly 10^{-5} for trace phases to 0.06 for dominant ones), which is why one temperature for all phases would not work. A phase that never occurs gets $T_0 = 0$; for the division in Eqs. (9)–(12) it is replaced by 10^{-6} times the smallest non-zero T_0, which makes its smoothing negligible.

Annealing. During an episode of N epochs, the temperature is $T = a\,T_0$, with a decreasing linearly from a_{start} to a_{end}:

$$a = a_{\text{start}} + (a_{\text{end}} - a_{\text{start}})\,\frac{\text{epoch}}{N-1}, \qquad \text{epoch} = 0, \ldots, N-1. \tag{15}$$

Early in training a large T gives every phase a wide, smooth transition region and well-behaved gradients. As T shrinks, $\tilde{n}$ approaches $\hat{n} = \max(g, 0)$ and the model is trained on its actual output. The schedule restarts in every episode that enables it (`boundary_temperature` in the recipe). Recipes typically chain two episodes, $a : 1 \to 0.05$ and then $a : 0.05 \to 10^{-4}$. All arithmetic involving g/T, and the mole heads that produce g, run in 32-bit floats even under mixed precision, because late in the schedule g/T would overflow or lose precision in 16-bit formats.

5 The training loop

Episodes. A recipe is a sequence of episodes, each with its own number of epochs, learning-rate schedule, frozen modules and loss-weight overrides. The continuous recipes run:

1. tune the encoder and chemistry heads, with the mole heads frozen and $\lambda_n = \lambda_s = 0$;
2. tune the mole heads, with the encoder and chemistry heads frozen and $\lambda_x = 0$;
3. train everything jointly, with annealing on and $\lambda_b > 0$.

Optimiser and input noise. AdamW (with AMSGrad and $\epsilon = 10^{-4}$), with weight decay set per module: encoder `lowWD`, chemistry heads `highWD`, mole heads none. Training inputs can be perturbed with multiplicative Gaussian noise, $[u, b] \leftarrow [u, b]\,(1+\sigma\xi)$ with $\xi \sim \mathcal{N}(0, 1)$; the perturbed b is also the bulk target in L_b.

Validation, checkpointing and early stopping. Before the first epoch, the incoming model is scored on the validation set; that score is the one to beat. After each epoch the validation loss (Eq. 4, at that epoch's T) is computed, and the model is saved whenever it improves on the best so far. Training stops after `early_stopping_patience` epochs without improvement, and the best saved model is restored.

Adaptive regularisation. After each epoch, if the validation loss exceeds 1.02 times the training loss, the dropout rate (outside the mole heads) is raised one step up to its ceiling; once at the ceiling, the input noise σ is raised instead. If the validation loss is below the training loss, dropout (or, once dropout is zero, σ) is lowered one step.

Remark on checkpoint selection under annealing. The validation loss is evaluated at a different T each epoch. Both the smoothing error ($\leq T \ln 2$) and $\tilde{n}$ for absent phases shrink with T, so the validation loss tends to fall as the schedule progresses regardless of model quality. Checkpoint selection and early stopping therefore favour later epochs, and the starting score (computed at the first epoch's T) is not directly comparable with later ones.